2026-9-18 Linux网络--TCP可靠传输与流量控制

- 1. TCP 可靠传输与流量控制
- 2. TCP 可靠性的含义
  - 2.1 确认应答
    - 2.1.1 历史报文可靠性
    - 2.1.2 数据丢失与 ACK 丢失
    - 2.1.3 重复与重传副本
  - 2.2 序号与累计确认
    - 2.2.1 字节序号
    - 2.2.2 确认号表示连续前缀
    - 2.2.3 序号承担的职责
  - 2.3 超时重传
    - 2.3.1 RTO
    - 2.3.2 快速重传
- 3. 发送缓冲区与滑动窗口
  - 3.1 为什么需要发送缓冲区
    - 3.1.1 未确认数据不能删除
    - 3.1.2 逻辑数组与物理队列
  - 3.2 窗口的四个区域
  - 3.3 窗口边界
    - 3.3.1 左右边界
    - 3.3.2 左边界不会后退
  - 3.4 窗口的正常推进
  - 3.5 窗口大小变化
- 4. 基于窗口的丢包处理
  - 4.1 接收端等待连续前缀
  - 4.2 最左侧丢失
  - 4.3 中间丢失
  - 4.4 最右侧丢失
  - 4.5 ACK 丢失
- 5. 流量控制
  - 5.1 接收窗口
  - 5.2 零窗口
  - 5.3 窗口扩大因子
  - 5.4 窗口与 MSS
- 6. 与拥塞控制和连接管理的边界

- 7. 小结

1. TCP 可靠传输与流量控制

本篇解决 TCP 数据面的两个问题：

1. 在会丢包、乱序和重复的网络中，如何确认数据已经到达并按序交付；
2. 如何根据接收方剩余空间控制发送量，避免把对端缓冲区压垮。

内容主线是：确认应答与超时重传 → 发送缓冲区 → 滑动窗口 → 丢包恢复 → 接收窗口与零窗口。

2. TCP 可靠性的含义

2.1 确认应答

2.1.1 历史报文可靠性

发送方发出数据后，只有收到接收方 ACK，才能确定这段历史数据已经被对端的 TCP 层按序接收。

TCP 的可靠不是“最新发出但尚未确认的数据一定已经到达”，而是：

- 收到确认的数据，可以视为对端已经按序接收；
- 未确认数据必须保留，以便丢失后重传；
- ACK 本身不需要再被 ACK，否则会形成无限递归；
- 网络链路是否真正送达由网络状态决定，TCP 提供的是检测、确认和补发机制。

2.1.2 数据丢失与 ACK 丢失

发送方超时没有收到 ACK，可能面对两种情况：

- 数据丢失，接收方从未收到；
- 数据已到达，但 ACK 丢失。

从发送方视角看，两者都是“没有收到确认”，无法直接区分。为了结束永久等待，只能设置超时并重传未确认数据。

2.1.3 重复与重传副本

如果 ACK 丢失后发送方重传，接收方可能再次收到相同序号的数据。接收方应识别重复部分并丢弃，再发送当前累计确认。

因此，发送出去但尚未确认的数据必须继续保存在发送缓冲区。提前删除会导致真正的数据丢失无法恢复。

2.2 序号与累计确认

2.2.1 字节序号

TCP 对字节流中的每个字节编号。数据段中的 `seq` 表示该段第一个数据字节的位置。

```
发送：seq = 1001，携带 1000 字节
下一段：seq = 2001
```

初始序号通常随机选择，不固定从 1 开始。32 位序号会回绕，但接收端仍在当前连接的字节流上下文中解释。

2.2.2 确认号表示连续前缀

确认号表示下一个期望收到的字节序号。`ack=2001` 的含义是 2001 之前的数据已经按序收到。

累计确认可以容忍部分 ACK 丢失：

```
收到 ACK 2001、ACK 3001、ACK 5001
即使 ACK 4001 丢失，
ACK 5001 仍证明 5001 之前的全部数据已经按序到达。
```

如果接收端中间存在缺口，就不能越过缺口把确认号推进到后面。

2.2.3 序号承担的职责

序号同时用于：

- 识别重复数据；
- 按序重排；
- 推进累计确认；
- 判断重传范围；
- 推进滑动窗口；
- 定位接收端在缺口之后暂存的数据。

2.3 超时重传

2.3.1 RTO

发送方为未确认数据启动重传计时器。超过 RTO 仍未收到确认，就重新发送对应数据。

RTO 需要根据往返时延动态估计：

- 设置过短会把正常延迟误判为丢包，产生大量重复报文；
- 设置过长会让真实丢包恢复缓慢；
- 连续超时时通常采用指数退避，逐步拉长等待时间。

用“500 ms 后翻倍”是理解退避的例子；真实数值由内核算法和网络测量决定。

2.3.2 快速重传

接收端收到后续数据但缺少中间某个缺口时，会反复发送当前期望的确认号。发送方连续收到重复 ACK，可以判断缺口可能丢失，并提前重传。

经典 TCP 常以“三个额外重复 ACK”作为快速重传触发条件。SACK 可以进一步告诉发送方具体缺少哪些区间，减少无效重传。

快速重传是效率优化，超时重传仍是最后兜底。

3. 发送缓冲区与滑动窗口

3.1 为什么需要发送缓冲区

3.1.1 未确认数据不能删除

TCP 允许连续发送多个数据段。发送出去后，在一段数据被累计确认覆盖之前，发送端必须保留副本。

```
应用写入
  -> TCP 发送缓冲区
  -> 分成多个报文段发送
  -> ACK 覆盖之前继续保存
  -> 得到确认后释放
```

3.1.2 逻辑数组与物理队列

逻辑上可以把发送缓冲区看成一维字节数组，每个字节都有序号。实际内核可能使用 `sk_buff` 队列和分片结构管理数据。

滑动窗口描述发送缓冲区中当前允许发送的一段范围。`start_win` 和 `end_win` 是理解模型的边界，而不是要求内核一定使用两个普通整数保存。

3.2 窗口的四个区域

区域	数据状态	是否可发送	是否可删除
已确认区域	已被累计 ACK 覆盖	已发送	可以释放
已发送未确认	在途等待 ACK	已发送	必须保留
窗口内未发送	已被窗口允许	可以发送	暂不删除
窗口外数据	无发送资格	不可发送	继续等待

需要注意：“在窗口内”不等于“已经发送”，“已经发送”也不等于“可以删除”。只有累计确认覆盖该字节之后，发送缓冲区副本才可以释放。

3.3 窗口边界

3.3.1 左右边界

简化模型中：

```
窗口左边界 = 最早未确认字节序号
窗口右边界 = 左边界 + 对端通告的接收窗口
实际可发送 = min(窗口范围, 拥塞窗口, 当前待发送数据量)
```

接收端发送的窗口字段代表自身剩余接收空间，因此发送方每个窗口的右边界都以对端最新通告为依据。

3.3.2 左边界不会后退

累计确认只会向前推进，窗口左边界不会回到旧序号。窗口大小可以因接收端处理变慢而缩小，这与“整个窗口向左滑动”不是一回事。

3.4 窗口的正常推进

假设当前窗口是 `[1001, 5001)`，接收端返回：

```
ack = 2001
win = 4000
```

窗口更新为：

```
新左边界 = 2001
新右边界 = 2001 + 4000 = 6001
```

1001 到 2000 已确认并可从发送缓冲区释放；5001 到 6000 如果已有数据，现在可以发送。

收到确认	左边界	右边界	窗口大小
初始	1001	5001	4000
ACK 2001	2001	6001	4000
ACK 3001	3001	7001	4000
ACK 4001	4001	8001	4000

3.5 窗口大小变化

接收端应用读取速度与数据到达速度匹配时，窗口可以不变；处理慢时窗口缩小；一次读走大量数据时窗口变大；缓冲区占满时窗口变为 0。

窗口形式上看可以收缩，但已确认前缀不能倒退。发送方必须保存未确认数据，直到累计确认覆盖。

4. 基于窗口的丢包处理

4.1 接收端等待连续前缀

如果收到 1001、2001、4001，却缺少 3001，接收端不能把 4001 提前交给应用。后续数据可以暂存，但确认号最多推进到缺口位置。

后续报文不断到达时，接收端会发送相同的当前确认号，形成重复 ACK。

4.2 最左侧丢失

发送 1001-2000、2001-3000、3001-4000、4001-5000，其中 1001-2000 丢失。接收端会持续确认 `ack=1001`。

发送端收到足够重复 ACK 后重传最左侧缺口。补齐后，接收端可以一次累计确认到 5001，窗口左侧推进到 5001。

4.3 中间丢失

如果缺少 3001-4000，接收端持续确认为 3001。从发送端维持连续前缀的角度看，这个问题仍转化为“当前左边界处存在缺口”。

发送端快速重传缺口后，接收端可将连续前缀一次推进到已缓存数据的最右端。

4.4 最右侧丢失

如果丢失的是窗口最右侧的最后一段，后面没有更多数据产生重复 ACK，发送方可能无法执行快速重传，只能等 RTO 到期后重传。

因此，快速重传并非所有丢包都能触发，超时重传必须始终保留。

4.5 ACK 丢失

如果数据到达而只有部分 ACK 丢失，后面的累计确认可能直接覆盖前面丢失的确认，发送方不需要仅因某个 ACK 丢失就重传已确认数据。

5. 流量控制

5.1 接收窗口

接收端的剩余空间决定窗口：

```
接收窗口 = 接收缓冲区总量 − 应用尚未读取的数据量
```

如果发送方忽略窗口，数据到达后可能因为缓冲区不足被丢弃，再触发重传，浪费链路和主机资源。

5.2 零窗口

接收缓冲区完全占满时，接收端通告 `win=0`，发送方停止发送新的普通有效载荷。

窗口更新报文可能丢失，因此发送方会周期性发送不携带业务数据的窗口探测，要求接收端重新报告窗口。接收方应用读走数据后，也可以主动发送窗口更新。

窗口探测和窗口更新并行存在，哪条信息先到就按哪条恢复发送。

5.3 窗口扩大因子

16 位窗口字段最大直接表示 65535 字节。连接建立时可以在 TCP 选项中协商窗口扩大因子：

```
实际窗口 = TCP 窗口字段值 * 2^窗口扩大因子
```

扩大因子在连接建立阶段协商，连接期间不能随意改变。

5.4 窗口与 MSS

窗口按字节计算，但 TCP 需要把数据拆成不超过 MSS 的报文段。一个 4000 字节窗口可能对应多个 TCP 报文，而不是一个 4000 字节数据段。

接收窗口、MSS/MTU 和拥塞窗口共同限制一次实际发送量。

6. 与拥塞控制和连接管理的边界

本篇已经解释接收窗口和发送滑动窗口。TCP 还要考虑共享网络是否拥堵，拥塞窗口与慢启动等内容见 2026-9-18 Linux网络--TCP拥塞控制与传输优化。

握手期间双方会交换初始窗口；四次挥手、TIME_WAIT、CLOSE_WAIT 和异常断线属于连接生命周期，见 2026-9-18 Linux网络--TCP连接管理与异常诊断。

7. 小结

```
确认序号 -> 推进可靠前缀
发送缓冲 -> 保存未确认数据
滑动窗口 -> 限定当前可发送范围
接收窗口 -> 防止压垮接收端
重复 ACK -> 暴露连续前缀中的缺口
快速重传 -> 尽快补缺口
超时重传 -> 无足够反馈时兜底
删除数据 -> 必须等待累计确认覆盖
```

TCP 的可靠性与效率在这里开始汇合：ACK 保证发送方知道结果，序号保证排序和去重，发送缓冲区保证可重传，窗口在接收能力范围内提高发送效率。拥塞控制进一步把网络承载能力纳入同一套发送范围计算。